

Add container pop methods that return the popped value

Document #: P3182R1
Date: 2024-07-16
Project: Programming Language C++
Audience: LEWG
Reply-to: Brian Bi
<bbi10@bloomberg.net>

Contents

- 1 Abstract
- 2 Revision history
 - 2.1 R1
- 3 Introduction
- 4 Prevalence
- 5 Reasons for the status quo
 - 5.1 Efficiency
 - 5.2 Exception safety
- 6 Solution
 - 6.1 Sequence containers
 - 6.1.1 `move` versus `move_if_noexcept`
 - 6.2 `std::stack` and `std::queue`
 - 6.3 `std::priority_queue`
- 7 Summary of design decisions
 - 7.1 Alternative names
 - 7.2 Exception safety
 - 7.3 Member or free functions
 - 7.4 Removal other than from the end
 - 7.5 Whether or not to delegate to underlying container
- 8 Wording
- 9 References

§ 1 Abstract

The `pop`, `pop_front`, and `pop_back` methods of the Standard Library containers and container adaptors do not return the value popped, which is a longstanding pain point for users of the Standard Library. I discuss how to implement versions of these methods so that objects are not dropped even when an exception occurs, and propose the inclusion of these methods in the Standard Library. I also discuss the special case of `std::priority_queue`, for which failure of a move operation will usually result in unrecoverable loss of data from `std::priority_queue` even under the status quo; therefore, I also propose the addition of `std::priority_queue::pop_value`.

§ 2 Revision history

§ 2.1 R1

- Added sequence containers
- Expanded discussion of exception safety
- Added discussion of other minor design decisions
- Added estimates of the amount of code that could benefit

§ 3 Introduction

The `pop` method appears in three container adaptors in the C++ Standard Library: namely, `std::stack`, `std::queue`, and `std::priority_queue`. The return type for each `pop` method is `void`; the popped object is destroyed and its value is not returned to the caller. Should the programmer wish to obtain the popped value, they must call the `top` or `front` method to obtain a reference to the object that is about to be popped, move the value of that object into some other object, and then call `pop`. Thus, idiomatic C++ requires three statements to accomplish what could be done with one if `pop` were to actually return the popped value:

```
class Widget {
    std::stack<int>      d_stack;
    std::array<int, 100> d_array;

    // ...

    int foo() {
        // other code ...

        // invalid:
        // return d_array[d_stack.pop()];

        const auto top = d_stack.top();
        d_stack.pop();
        return d_stack[top];
    }
};
```

```
}  
}
```

The fact that `pop` does not return the popped value is a source of widespread annoyance among C++ users and is bewildering to beginners, particularly those coming from other languages in which pop-like methods do return the popped value, such as Java (`LinkedList.poll`, `PriorityQueue.poll`, etc.), Python (`list.pop`), JavaScript (`Array.prototype.pop`), and so on.¹

In the first version of this paper, I proposed to add value-returning versions of the `pop` methods only for the container adaptors `std::stack`, `std::queue`, and `std::priority_queue` because my personal experience suggested that these methods would be most useful for implementing algorithms in which elements are unconditionally removed and processed in a LIFO, FIFO, or priority-based order. However, the consensus on the LEWG reflector was that it is rare for users to use `std::stack` and `std::queue` in preference to using sequence containers directly. Therefore, this revision also proposes the addition of `pop_front_value` and `pop_back_value` to all containers that have `pop_front` and `pop_back`, respectively.

§ 4 Prevalence

Of the roughly 94 million publicly available C++ header and source files available to search on `github.com`, I estimate that approximately 220,000 of them contain at least one occurrence of `std::stack::top` followed unconditionally by `std::stack::pop`, where the only code that runs between the two calls can be reordered after the `pop` with no change in meaning. The analogous estimate for `std::priority_queue` is 140,000. These estimates were obtained by examining the first 100 search results for the query `/top\(\).*(\n)?.*pop\(\)/ language:C++` and manually ruling out false positives such as conditional pops and assuming that they are representative of the 430,000 matching files reported by GitHub. The majority of these 360,000 files use stacks and priority queues in order to implement graph algorithms such as depth-first search and Dijkstra's algorithm.

Note that there are an estimated 840,000 files on GitHub that include the `<stack>` header. I don't think that the ratio of 220,000 to 840,000 should be taken too seriously because there is not a one-to-one correspondence between files that use `std::stack` and files that include the `<stack>` header. Nevertheless, I believe the results show that a significant minority of code that uses `std::stack` could benefit from a `pop_value` method.

Similarly, there are an estimated 240,000 files on GitHub that call `std::vector::back` followed unconditionally by `std::vector::pop_back` (and much smaller numbers for `std::deque`, `std::list`, and `std::string`), compared with 7,500,000 files that include the `<vector>` header. These results confirm an assertion I made on the Library Evolution reflector that, although `std::vector` is used far more than the container adaptors, the peek-and-unconditionally-pop operation is a far more significant use case for container adaptors than containers.

§ 5 Reasons for the status quo

§ 5.1 Efficiency

The `std::stack`, `std::queue`, and `std::priority_queue` container adaptors, like many other components of the C++ Standard Library, were inspired by SGI's Standard Template Library (STL). The STL documentation [STL] explains why separate `top/front` and `pop` methods are provided: `pop` cannot return by reference since the reference would dangle, but returning by value would impose on the user the cost of copying the object. Because imposing this cost would be undesirable, I do not propose to change the existing `pop` methods to return the popped value, but instead propose a new `pop_value` method. In modern C++, the cost of calling `pop_value` will typically be minimal due to move semantics. For example, here is a possible implementation of `std::stack::pop_value`:

```
value_type pop_value() {  
    value_type x = move(top());  
    pop();  
    return x;  
}
```

In the above implementation, NRVO allows `x` to be treated as an alias for the returned object (§11.9.6 [class.copy.elision]²p1.1 of the Standard). GCC always performs this optimization by default and Clang performs it at `-O1` and higher (but both permit the user to explicitly disable it using the `-fno-elide-constructors` flag). When NRVO is performed, a single call to `value_type`'s move constructor occurs when `pop_value` is called. When NRVO is not performed, the move constructor is called twice (§7.5.4.2 [expr.prim.id.unqual]).

The considerations above also apply to the sequence containers (other than `std::array`, which cannot be popped from because its size is fixed): the introduction of new methods alongside the existing `pop_front` and `pop_back` methods, together with the use of move semantics, would allow users to pop and get the popped value at the same time, without incurring the performance penalties outlined by the SGI STL documentation.

§ 5.2 Exception safety

The more well-known reason for `pop` not returning the popped value is the issue of exception safety. In the implementation of `pop_value` above, although NRVO will often be performed, it is not guaranteed. Now suppose that NRVO is not performed. Then, the returned object is initialized by moving from `x`. If this move operation exits via an exception, it is impossible to provide the strong exception safety guarantee even if `value_type`'s move constructor provides it. In order to restore the state of the stack or queue as it existed prior to the call to `pop_value`, the popped object would need to be put back at the top of the stack or the front of the queue. This operation cannot be accomplished without calling the move constructor again (*i.e.*, the very operation that has

just failed). Therefore, the only choice is to destroy the popped object and propagate the exception. The object popped is thus “dropped” and its value is lost forever.

§ 6 Solution

§ 6.1 Sequence containers

It is possible to implement `std::vector<T>::pop_back_value` so that it provides the same exception safety guarantee as moving from back and then calling `pop_back` upon success, and, therefore, does not drop elements when `T`’s move constructor throws. The following is a proof-of-concept implementation that works for any sequence container that provides `pop_back`. The implementation for `pop_front_value` is very similar but uses `front()` and `pop_front()` instead of `back()` and `pop_back()`.

```
value_type pop_value() {
    struct Guard {
        vector* __this;
        bool failure = false;
        ~Guard() noexcept(false) {
            if (!failure) {
                __this->pop_back();
            }
        }
    } guard{this};
    try {
        return move(back());
    } catch (...) {
        guard.failure = true;
        throw;
    }
}
```

In this implementation, only a single move occurs, even if the compiler declines to perform NRVO. Thus, there is only a single point where we must pay attention to the possibility of a move constructor exiting by throwing an exception. Note that it is the caller’s responsibility to ensure that the popped value is not lost *after* `pop_back_value` returns. For example:

```
void foo(std::vector<Widget>& s) {
    Widget w = s.pop_back_value();
    // do something with `w` ...
    w = s.pop_back_value();
}
```

In the above code, due to guaranteed copy elision [P0135R1], the object stored in `s` is moved directly into the local variable `w` when `w` is initialized. If this move operation succeeds, the moved-from object is popped from `s`. If the move operation fails by throwing an exception, a flag is set and the destruction of `guard` does not attempt to remove the object from `s`. Therefore, if `value_type`’s move constructor provides the strong exception safety guarantee, then the `pop_back_value` implementation above

provides the same guarantee. If `value_type`'s move constructor provides the basic exception safety guarantee, then `pop_back_value` provides the basic exception safety guarantee in that the vector remains in a valid state, and additionally guarantees that the item at the back of the vector is not lost (though its original value might not be recoverable).

However, if the move assignment operator of `Widget` throws after the second call to `pop_back_value`, then, even if that operator provides the strong exception safety guarantee, the temporary object materialized from `s.pop_back_value()`, which still holds the popped value, will be destroyed and the value will be lost. If the caller of `pop_back_value` is able to recover from a move operation that throws, then the caller can avoid dropping the popped value by declaring another local variable to hold the popped value before calling the move assignment operator. Of course, such concerns need not trouble a user who has provided `Widget` with a non-throwing move assignment operator.

Note that the above implementation is simply a proof-of-concept; one optimization that I anticipate library implementors wanting to make (but that is not necessary to specify in the Standard) is that if `std::is_nothrow_move_constructible_v<value_type>` is true, then the try/catch block and the failure variable can be omitted.

§ 6.1.1 `move` versus `move_if_noexcept`

In cases where `value_type`'s move constructor is not `noexcept`, there is a question of whether the implementation of `pop_(front|back)_value` should call `std::move` or `std::move_if_noexcept`.

The most well-known usage of `std::move_if_noexcept` is by `std::vector` when reallocating. Copying, rather than moving, objects with throwing move constructors during reallocation enables `std::vector` to provide the strong exception safety guarantee for some operations that may reallocate.

For the proposed `pop_(front|back)_value` methods of sequence containers, it is also possible to provide the strong exception guarantee unconditionally (*i.e.*, regardless of what `value_type` is) by using `std::move_if_noexcept` instead of `std::move`. However, using `std::move_if_noexcept` also has a cost because it results in possibly unexpected and expensive copies for reasons that are unfamiliar even to many experienced C++ programmers. For example, consider an implementation in which `std::deque` does not have a `noexcept` move constructor. If `std::vector::pop_back_value` uses `std::move_if_noexcept`, then on such an implementation:

- A call to `std::vector<std::deque<T>>::pop_back_value` will not compile if `T` is a move-only type such as `std::unique_ptr<U>`.
- A call to `std::vector<std::deque<T>>::pop_back_value` will perform a copy in order to initialize each returned `std::deque<T>` object.

In the first case, a beginner will find it difficult to understand why seemingly reasonable code does not compile. In the second case, a beginner will write code that performs worse

than expected, and may find it very difficult to even recognize the possibility that the performance is due to the implementation making an implicit choice to perform copies.

It has been suggested that failing to provide strong exception safety creates a trap for beginners because the value of the popped item may not be recoverable in the event that the popped item's move constructor only provides the basic exception safety guarantee. I believe that the current Standard Library is already a trap for beginners in that sense, because it contains a large number of functions that are either not strongly exception-safe or that provide strong exception safety only under certain circumstances. For example, most functions that pop a single element are strongly exception-safe, but `std::priority_queue<T>::pop` is not; functions that insert a single element at the end of a `std::vector` or `std::deque` (but not the middle) are strongly exception-safe if the element type is copyable or has a non-throwing move constructor, but not otherwise; and functions that insert or remove several objects at once are usually not strongly exception-safe, but they are in the case of `std::list`. In any moderately complex application that makes use of the Standard Library, it is extremely unlikely that a beginner would, simply through sheer luck, avoid losing data along all paths in which a Standard Library function exits via an exception; the Standard Library is simply too riddled with traps for anyone to dodge all of them except through constant vigilance. In order to write a correct program whose correctness depends on the strong exception safety guarantees of the Standard Library, it is necessary to investigate every Standard Library function individually, understand the circumstances in which it is strongly exception-safe, and use (or avoid) it accordingly.

Programmers who are aware of the fact that a variety of different exception safety guarantees are present in the Standard Library, and who have the level of understanding required to avoid calling Standard Library functions in ways that are not strongly exception safe, will not be burdened by the presence of `pop_value`, `pop_front_value`, and `pop_back_value` methods; their *modus operandi* is necessarily to avoid any new functions until they have researched their exception safety guarantees. All other programmers are already not writing correct programs when their logical correctness depends on the strong exception safety guarantees in the Standard Library; if the possible loss of data for such programmers is untenable, they must be preventing it using drastic measures such as disabling exceptions or completely avoiding the usage of the Standard Library.

Therefore, the cost/benefit analysis for beginner C++ programmers favors the use of `std::move` instead of `std::move_if_noexcept`: the former avoids confusing compilation errors and degradation of performance, and does not significantly worsen the status quo in which beginners who use the Standard Library together with exceptions cannot avoid data loss except in the unlikely event that they carefully research each Standard Library function's exception safety guarantees prior to using it.

Experts who desire strong exception safety would be better served by the implicit use of `std::move_if_noexcept` than `std::move`, but there are also experts for whom strong exception safety is not important. To evaluate the impact of this design decision on

experts, we need to consider the cost to the expert to obtain the desired behavior when a Standard Library function provides the opposite one. If `pop_back_value` uses `std::move` but an expert wants strong exception safety, they can simply continue writing the sequence of `move_if_noexcept(back())` followed by `pop_back()` that they are likely currently using. If `pop_back_value` uses `std::move_if_noexcept` but an expert wants an unconditional move, they can implement a non-member `pop_back_value` that moves unconditionally using one of the two approaches above. The burden is not huge in either case, but I think there are more experts who don't need strong exception safety than ones who do.

Lastly, I believe there are two differences from vector reallocation that suggest that `std::move` is a better choice in the `pop_back_value` case than in the `push_back` case:

1. `pop_back_value` uses only a *single* call to the move constructor; therefore, if `pop_back_value` uses `std::move`, it inherits the exception safety guarantee from `T`'s move constructor: in other words, if `T`'s move constructor provides the strong exception safety guarantee, then so does `pop_back_value`. The same is not true for vector reallocation.
2. In cases where the user needs strong exception safety unconditionally (*i.e.*, even when `T`'s move constructor is not known to provide the strong exception safety guarantee), there is a simple workaround discussed above. In contrast, if vector reallocation were to use `std::move`, it would be much more inconvenient for the user to recover the strong exception safety guarantee.³

§ 6.2 `std::stack` and `std::queue`

To implement `std::stack::pop_value`, there are two options. One is to delegate the entire operation to the underlying container, *e.g.*, the body can simply be `return c.pop_back_value();`. The other is to call `return move(top());` and then call `pop()` in the destructor of a guard variable if the former was successful, similarly to the possible implementation shown above for `std::vector::pop_back_value()`.

When `std::stack` wraps a Standard Library sequence container (as it does by default), there is no difference in behavior between these two implementation strategies for the stack's `pop_value` method. They differ in the case where `std::stack` wraps a user-defined container that does not provide `pop_back_value` or, perhaps, provides only `pop_back_value` and not `back` (which could be the case for a concurrent data structure).

I propose that the `pop_value` method of `std::stack` call the underlying container's `pop_back_value` methods for the following reasons:

- If `pop_back_value` is provided by the underlying container but `back` itself is not, then a `pop_value` that calls `pop_back_value` will be the only way to remove and process the values stored in the `std::stack`.

- If the container supports both `pop_back_value` and `back` but with different behavior, then a container adaptor `pop_value` that calls the former follows the principle of least surprise.

Similarly, I propose that `std::queue::pop` call the underlying container's `pop_front_value` method.

The strategy of calling `top()` followed by `pop()` raises an additional design question: what should happen when `pop()` throws an exception because the underlying container's `pop_back` method throws an exception? Such a scenario cannot arise when the underlying container is a Standard Library container⁴, but this requirement has never been imposed on user-defined containers that are used to instantiate a container adaptor. Revision 0 of this paper [P3182R0] analyzed several options for dealing with a throwing `pop_back`, which are not repeated in this revision.

It is conceivable for `std::stack::pop_value` to call `c.pop_back_value()` when that expression is well formed, and otherwise fall back to an implementation that calls `top()` and `pop()`. Having `pop_value` unconditionally call `c.pop_back_value()` (and be unavailable when `c` does not provide `pop_back_value`) leaves open the possibility of introducing such a fallback in the future if consensus can be achieved on how to deal with a throwing `pop_back`.

§ 6.3 `std::priority_queue`

The case of `std::priority_queue` is very different. It is not possible to move from the top of a heap without compromising the heap's invariant, so `std::pop_heap` must be called on the underlying container before the returned object can be initialized. In general, `std::pop_heap` will use a number of move operations that is approximately logarithmic in the current size of the heap, and if any such operation besides the first throws an exception, the heap will be left in an invalid state. This risk also exists in the status quo: although the user will call `top` first and `pop` afterward, a failure in `std::pop_heap` during the second step will make it impossible to extract any further data safely from the `priority_queue`. Therefore, `std::priority_queue` is inherently unreliable in the presence of throwing moves. For this reason, I think that the possibility of throwing moves is no rationale for omitting `pop_value` from `std::priority_queue`. Furthermore, if a move operation were to throw, then it is more likely for the exception to occur during the `pop_heap` step than the initialization of the return object. Therefore, I consider it unnecessary to provide a guarantee about the state of the `priority_queue` even if the final move throws; the caller cannot tell whether the exception originated from the `pop_heap` step, and therefore, cannot rely on any such guarantee. For example, a conforming implementation need not check whether the final move succeeded:

```
value_type pop_value() {
    pop_heap(c.begin(), c.end(), comp);
    struct Guard {
        priority_queue* __this;
        ~Guard() noexcept(false) {
```

```

        __this->c.pop_back();
    }
} guard{this};
return move(c.back());
}

```

§ 7 Summary of design decisions

This section summarizes design decisions that may be controversial or otherwise merit special consideration from the LEWG. Some of these design decisions have been discussed in more depth earlier in this paper.

§ 7.1 Alternative names

For the container adaptors, the name `pop_value` was suggested by Lauri Vasama. Other possible names include `pop_and_get` and `pop_and_return`. For the sequence containers, I propose the names `pop_front_value` and `pop_back_value` in analogy with `pop_value`. Arthur O'Dwyer has implemented similar methods named `displace`, `displace_back`, `displace_top`, etc. I consider `pop_value` and its variants to be a slightly better name than `displace` and its variants because, when a container has both a `pop` and a `displace` method, it will not be obvious to the user which one returns a value and which does not. However, one argument in favor of `displace` is that constructions such as `displace_back` are less awkward than `pop_back_value`. Indeed, a user might forget whether the word `back` or the word `value` goes first. The reason I propose `pop_front_value` instead of `pop_value_front` is that, by placing the word `value` at the end rather than between `pop` and `front`, we emphasize that `pop_front_value` does the same thing as `pop_front`, with the only difference being that a value is returned. I admit that this is not a particularly compelling reason, and I am not opposed to the name `pop_value_front`.

§ 7.2 Exception safety

This paper currently proposes the usage of `std::move` rather than `std::move_if_noexcept`, as discussed in Section 5.1.2. If the LEWG polls to encourage the usage of `std::move_if_noexcept` instead, this paper will be updated to propose that.

A suggestion was made on the LEWG reflector that the Standard Library could benefit from a trait that indicates whether the move construction of a given type is strongly exception-safe. The trait would automatically be true for all scalar types and all types that are nothrow move constructible, but other types could opt into the trait, such as a `std::list` implementation that always attempts to allocate a head node for the moved-from object first, before proceeding to an operation that cannot fail (*i.e.*, swapping the head node pointers of the source and destination). If such a trait were available, then `pop_value` methods (other than for `std::priority_queue`) could be made strongly exception safe by using the move constructor when the move constructor is strongly exception-safe, and the copy constructor otherwise.

This paper does not actually propose a trait for strong exception safety of move construction. However, the possibility of such a direction suggests a third design option for this paper, namely to provide the proposed methods only when the element type is nothrow move constructible. Such a design, despite having the unfortunate effect of making the methods unavailable in certain cases in which users would expect them to be available, would maximize forward compatibility with a future design based on the trait described above.

§ 7.3 Member or free functions

It was suggested on the LEWG reflector that I explain why I propose that the `pop_value` functions be added as member functions rather than free functions. The LEWG does not currently have a policy that would help to make this decision. I note, however, that in the case of the container adaptors, the proposed semantics require at least protected access to the container adaptor, since a `pop_(front|back)_value` method must be called on the protected member `c`. Generally, operations on Standard Library types that cannot be implemented correctly without recourse to the non-public interface are made member functions, except in the case of operators and swap functions, which are sometimes friends. In the case of the sequence containers, non-public access is not required, and Meyers's algorithm [meyers00] would therefore indicate a non-friend free function. However, Meyers's algorithm is generally not followed by the Standard Library. While I would personally support a LEWG policy that is more in line with Meyers's algorithm, one does not currently exist, so I propose member functions for consistency with the rest of the container operations in the Standard Library.

§ 7.4 Removal other than from the end

Although methods that remove an element from the middle of a sequence container are not named with `pop`, it is conceivable that a user might want a variation of `erase` that returns the element removed. Similarly, a user might want such methods for associative and unordered associative containers. I do not currently propose `erase_value` methods because I believe that methods that return the removed value are most often useful for implementing algorithms in which elements are unconditionally removed (*i.e.* without first looking at the value) and processed in a LIFO, FIFO, or priority-based order. I do not take a position here on whether `erase_value` methods should be added to the Standard Library by a separate proposal.

§ 7.5 Whether or not to delegate to underlying container

As discussed in Section 5.2, this paper proposes that container adaptors' `pop_value` methods delegate to the underlying containers' `pop_back_value` and `pop_front_value` methods instead of calling `top` followed by `pop_back` or `front` followed by `pop_front`.

§ 8 Wording

The proposed wording is relative to [N4981].

Modify §9.4.5 [dcl.init.list]p1:

[...] List-initialization can occur in direct-initialization or copy-initialization contexts; list-initialization in a direct-initialization context is called *direct-list-initialization* and list-initialization in a copy-initialization context is called *copy-list-initialization*. Direct-initialization that is not list-initialization is called *direct-non-list-initialization*; copy-initialization that is not list-initialization is called *copy-non-list-initialization*.
[...]

Modify §23.4.3.1 [basic.string.general]p3:

```
// ...  
constexpr void pop_back();  
constexpr charT pop_back value();  
  
// ...
```

Modify §24.2.4 [sequence.reqmts] by inserting the following after paragraph 112:

```
a.pop_front_value()
```

Result: T

Preconditions: a.empty() is false and T is *Cpp17MoveConstructible*.

Effects: Copy-non-list-initializes the return object from std::move(front()), then calls pop_front.

Remarks: Required for deque, forward_list, and list.

Modify §24.2.4 [sequence.reqmts] by inserting the following after paragraph 116:

```
a.pop_back_value()
```

Result: T

Preconditions: a.empty() is false and T is *Cpp17MoveConstructible*.

Effects: Copy-non-list-initializes the return object from std::move(back()), then calls pop_back.

Remarks: Required for basic_string, deque, list, and vector.

Modify §24.3.8.1 [deque.overview]p2:

```
// ...  
void pop_front();  
T pop_front value();  
void pop_back();
```

```
T_pop_back_value();  
  
// ...
```

Modify §24.3.9.1 [\[forward.list.overview\]](#)p3:

```
// ...  
void pop_front();  
T_pop_front_value();  
  
// ...
```

Modify §24.3.10.1 [\[list.overview\]](#)p2:

```
// ...  
void pop_front();  
T_pop_front_value();  
void push_back(const T& x);  
// ...  
void pop_back();  
T_pop_back_value();  
  
// ...
```

Modify §24.3.11.1 [\[vector.overview\]](#)p3:

```
// ...  
constexpr void pop_back();  
constexpr T_pop_back_value();  
  
// ...
```

Modify §24.3.12.1 [\[vector.bool.pspc\]](#)p1:

```
// ...  
constexpr void pop_back();  
constexpr bool_pop_back_value();  
  
// ...
```

Modify §24.6.6.1 [\[queue.defn\]](#)p1:

```
// ...  
void pop() { c.pop_front(); }  
value_type pop_value();  
void swap(queue& q) noexcept(...)  
// ...
```

Add the following paragraphs to the end of §24.6.6.4 [\[queue.mod\]](#):

```
value_type pop_value();
```

Constraints: `c.pop_front_value()` is a valid expression that is convertible to `value_type`.

Returns: `c.pop_front_value()`.

Modify §24.6.7.1 [[priority_queue.overview](#)]p1:

```
// ...  
void pop();  
value_type pop_value();  
void swap(priority_queue& q) noexcept(...)  
// ...
```

Add the following paragraphs to the end of §24.6.7.4 [[priority_queue.members](#)]:

```
value_type pop_value();
```

Constraints: `c.pop_back_value()` is a valid expression that is convertible to `value_type`.

Effects: Calls `pop_heap(c.begin(), c.end(), comp)`.

Returns: `c.pop_back_value()`.

Modify §24.6.8.2 [[stack.defn](#)]p1:

```
// ...  
void pop() { c.pop_back(); }  
value_type pop_value();  
void swap(stack& s) noexcept(...)  
// ...
```

Add the following paragraphs to the end of §24.6.8.5 [[stack.mod](#)]:

```
value_type pop_value();
```

Constraints: `c.pop_back_value()` is a valid expression that is convertible to `value_type`.

Returns: `c.pop_back_value()`.

§ 9 References

[meyers00] Scott Meyers. 2000-02-01. How Non-Member Functions Improve Encapsulation.

<https://www.drdobbs.com/cpp/how-non-member-functions-improve-encapsu/184401197>

[N4981] Thomas Köppe. 2024-04-16. Working Draft, Programming Languages — C++.

<https://wg21.link/n4981>

[P0135R1] Richard Smith. 2016-06-20. Wording for guaranteed copy elision through simplified value categories.

<https://wg21.link/p0135r1>

[P3182R0] Brian Bi. 2024-04-16. Add `pop_value` methods to container adaptors.

<https://wg21.link/p3182r0>

[STL]

<http://web.archive.org/web/20010221200527/http://www.sgi.com/tech/stl/stack.html>

1. These methods also provide (weak) evidence that “pop and return the popped value” is a useful operation. In fact, I can’t think of any reason why, in any language other than C++, a pop method would ever be intentionally designed to return `void` (or its equivalent). ↩
2. All citations to the Standard are to working draft N4981 unless otherwise specified. ↩
3. The technique for doing so would be: suppose one wishes to add an element to the end of the vector. If `T` is nothrow-move-constructible, or if `size() < capacity()`, then just call `push_back` normally. Otherwise, create a new empty vector, reserve some multiple of the original vector’s capacity, copy-insert the old vector’s elements into it followed by the new element, swap the new vector with the old one, and then destroy the new one. ↩
4. The `pop_front` and `pop_back` methods of the Standard Library’s sequence containers never throw an exception (§24.2.2.2 [container.reqmts]p66.3). ↩